

R 디버깅 가이드

R 프로그래밍 with workflow design



예제 함수

IRLS 예제 함수의 목적

- 목표: 로지스틱 회귀모형을 직접 구현하여 디버깅을 연습함
- 핵심 알고리즘: IRLS (Iteratively Reweighted Least Squares)
- 활용 이유
  - 수치해석 루틴과 반복 알고리즘 디버깅 연습
  - `solve()`, `%*%`, `diag()` 등 행렬 계산의 이해 필요

로지스틱 함수와 WLS (가중 최소제곱) 함수 정의

```r
logistic = function(a) {
  return(1 / (1 + exp(-a)))
}

wls = function(X, y, W = NULL) {
  if (is.null(W))
    W = diag(1, length(y))
  return(solve(t(X) %*% W %*% X, t(X) %*% W %*% y))
}
```

glm_irls 함수 정의

```r
glm_irls = function(X, y, max_iter = 1000, epsilon = 1e-8){
  p = ncol(X)
  beta_hat = beta_hat_old = rep(1, p)
  diff = 1 ; iter = 1

  while((diff >epsilon) & (iter <max_iter)) {
    iter = iter + 1
    beta_hat_old = beta_hat

    y_hat = logistic(X %*% beta_hat)
    y_hat_prod = y_hat * (1 - y_hat)
    W = diag(as.vector(y_hat_prod))
    z = (y-y_hat) /y_hat_prod
    beta_hat = beta_hat + wls(X,z,W)
    diff =sum((beta_hat-beta_hat_old)^2)
  }
  return(beta_hat)
}
```

예제 시뮬레이션 코드

```r
set.seed(923)
n = 1000
x1 = runif(n)
x2 = runif(n)
X = cbind(1, x1, x2)
beta = c(1, 3, -2)
y = rbinom(n, 1, logistic(X %*% beta))
beta_est = glm_irls(X, y)

as.vector(beta_est)
```

```
## [1]  1.075482  2.976426 -2.056812
```

```r
glm(y ~ x1 + x2, family = binomial())$coefficients
```

```
## (Intercept)          x1          x2
##    1.075482    2.976426   -2.056812
```

- glm() 함수와 동일한 결과 제공

`traceback()`을 사용한 디버깅

`traceback()` 함수란?

- `traceback()`은 에러 직후에 함수 호출 스택을 출력하여
- 어느 함수에서 에러가 발생했는지 추적할 수 있도록 도와줌
- 특히 중첩 함수 호출이 많은 경우 매우 유용함

예제: 실수로 잘못된 변수명 사용

```
wls = function(X, y, W = NULL) {
  if (is.null(W))
    W = diag(1, length(y))

  # 오타: W → Wt
  return(solve(t(X) %*% Wt %*% X, t(X) %*% Wt %*% y))
}
```

```
beta_est = glm_irls(X,y)
```

```
Error in wls(X, z, W): object 'Wt' not found
```

```
traceback()
```

```
3: solve(t(X) %*% Wt %*% X, t(X) %*% Wt %*% y) at #6
2: wls(X, z, W) at #19
1: glm_irls(X, y)
```

- `solve()` 함수 인자에 런타임 환경에 존재하지 않는 `Wt` 변수를 사용하려 했음을 짐작 가능

`browser()`를 사용한 디버깅

`browser()` 디버깅이란?

- `browser()`: R에서 지정한 줄에서 코드 실행을 일시 정지하고 디버깅 모드로 진입
- 디버깅 중 실행 환경을 탐색하고 변수 값 확인 가능
- 복잡한 로직 중 특정 조건에서만 멈추고 싶은 경우 매우 유용

browser() 삽입 예제

```
glm_irls = function(X, y, max_iter = 1000, epsilon = 1e-8){
  p = ncol(X)
  beta_hat = beta_hat_old = rep(1, p)
  browser()  # 디버깅 모드 진입
  diff = 1 ; iter = 1

  while((diff > epsilon) & (iter < max_iter)) {
    iter = iter + 1
    beta_hat_old = beta_hat

    y_hat = logistic(X %*% beta_hat)
    y_hat_prod = y_hat * (1 - y_hat)
    W = diag(as.vector(y_hat_prod))
    z = (y-y_hat) / y_hat_prod
    beta_hat = beta_hat + wls(X,z,W)
    diff = sum((beta_hat-beta_hat_old)^2)
  }
  return(beta_hat)
}
```

예제 실행

```
glm_irls(X, y, max_iter = 2)
```

- `max_iter = 2`로 설정 시 수렴하지 않음
- 디버깅 모드 없이 실행하면 문제의 원인을 파악하기 어려움

디버깅 모드 예시

```
 1  glm_irls = function(X, y, max_iter = 1000, epsilon = 1e-8)
 2  {
 3    p = ncol(X)
 4    beta_hat = beta_hat_old = rep(1, p)
 5
 6    diff = 1
 7    iter = 1
 8
 9    while((diff > epsilon) & (iter < max_iter))
10    {
11      iter = iter + 1
12      beta_hat_old = beta_hat
13
14      y_hat = logistic(X %*% beta_hat)
15      y_hat_prod = y_hat * (1 - y_hat)
16      W = diag(as.vector(y_hat_prod))
17      z = (y - y_hat) / y_hat_prod
18
19      beta_hat = beta_hat + wls(X, z, W)
20      diff = sum((beta_hat - beta_hat_old)^2)
21    }
```

Console | Terminal | Background Jobs

R 4.4.1 · ~/

Next | Continue | Stop

```
        W = diag(as.vector(y_hat_prod))
        z = (y - y_hat)/y_hat_prod
        beta_hat = beta_hat + wls(X, z, W)
        diff = sum((beta_hat - beta_hat_old)^2)
    }
    return(beta_hat)
}
Browse[1]> n
debug at #3: p = ncol(X)
Browse[1]> n
debug at #4: beta_hat = beta_hat_old = rep(1, p)
Browse[1]> n
debug at #6: diff = 1
Browse[1]>
```

Figure 1: 디버깅 모드 실행 화면

디버깅 모드 옵션

- `browser()` 실행 시 콘솔 프롬프트가 `Browse[1]>`로 바뀜
- 이 상태에서 아래 명령 사용 가능
  - `ls()`: 현재 환경의 변수 목록 확인
  - `print(x)`: 특정 변수 `x`의 값 확인
  - `n` 또는 `Enter`: 다음 줄 실행
  - `c`: 디버깅 종료 후 나머지 코드 실행
  - `Q`: 디버깅 종료 및 즉시 중단
- `browser()` 실행 시 디버깅 툴바도 활성화되어 코드 실행을 제어하고 변수 값을 확인할 수 있게 해

debug() 와 debugonce() 를 사용한 디버깅

debug() 함수

- 특정 함수에 대해 항상 디버깅 모드로 실행하도록 설정
- 함수의 가장 처음부터 한 줄씩 실행 가능
- 매번 함수가 호출될 때마다 디버깅을 원할 경우 사용

```
debug(glm_irls)                # glm_irls 함수 디버깅 설정
glm_irls(X, y)                 # 실행하면 자동으로 디버깅 모드 진입
undebug(glm_irls)              # 디버깅 해제
```

debugonce() 함수

- 단 한 번 디버깅 모드로 실행
- 두 번째 호출부터는 일반 실행
- 일시적으로 디버깅할 때 가장 자주 사용하는 방법

```
debugonce(glm_irls)
glm_irls(X, y)                  # 디버깅 모드로 실행
glm_irls(X, y)                  # 디버깅 없이 실행됨
```

요약

도구	디버깅 진입 시점	용도
`debug()`	함수의 **처음부터**	함수 전체 흐름을 확인하고 싶을 때
`browser()`	**지정한 위치**	특정 조건 혹은 시점에서 멈추고 싶을 때
`debugonce()`	처음 한 번만 `debug()` 실행	일시적 디버깅이 필요할 때